CT008 Live Database Integration Guide

Reading detector status and one-second readings from the SQLite database written by the CT008 Windows application

Document version: 1.0 **Date:** 22 September 2026
Applies to: CT008 application 1.9.0, database schema 3, MicroR Meter **Issued by:** EIC

1. Overview
2. Where the database is and how it is configured
3. Concurrency model and rules for readers
4. Schema reference
5. Detector identity, sessions and timestamps
6. Interpreting detector status
7. Reading new data: the cursor pattern
8. Building 5-second samples from 1-second readings
9. Data retention
10. Schema versioning and forward compatibility
11. Useful queries
12. Code examples (C# and Python)
13. Troubleshooting
14. Reference implementation

1. Overview

The CT008 Windows application connects to EIC MicroR meters over Bluetooth, polls each connected meter once per second, and records the readings. Alongside its CSV log files it writes every reading and every change of detector state to a single SQLite database. The database is designed to be read by a separate client application while it is being written, with no coordination beyond what SQLite itself provides.

The database holds two tables:

- **detector_status**: one row per detector ever seen, giving its identity, whether it is connected or logging, the logging session it belongs to, its response time setting, and a heartbeat that shows whether the CT008 instance behind it is still alive.
- **samples**: one row per reading, tagged with the detector serial number, the logging session, a per-session sequence number and a UTC timestamp, plus the measured values.

In the usual deployment one CT008 instance runs per detector. All instances on a PC write to the same database file, so a client sees every detector on that PC in one place. The database contract is identical whether one instance or several are writing.

2. Where the database is and how it is configured

Settings live in `CT008.exe.config` next to the application. The keys that matter to a client are:

Key	Default	Meaning
`SqliteEnabled`	`true`	Turns the database output on or off. A missing key means on.
`SqliteRetentionHours`	`24`	Readings older than this are deleted by the application once a minute. `0` keeps everything.

Key	Default	Meaning
`CsvEnabled`	`true`	CSV log output, independent of the database. Both can be on at once.

The database file is always `<CT008 folder>\Database\ct008-log.sqlite`, next to the application and apart from the `Export` folder that holds the CSV logs. Its location is not configurable in this release. Two companion files, `ct008-log.sqlite-wal` and `ct008-log.sqlite-shm`, appear next to it whenever the database is open. They are part of normal operation and must be left alone.

The application creates the folder, the file and the tables on first use. If the database is deleted while no CT008 instance and no client has it open, the next instance to start simply creates a fresh one. Delete all three files together, never the `.sqlite` file alone. A recreated database starts its reading ids again at 1; see section 7 for what that means for a stored cursor.

> **Local disk only.** The database uses SQLite's write-ahead log, which relies on shared memory between processes. It does not work across a network share, so the CT008 instances and the client must run on the same PC and the file must be on a local drive.

3. Concurrency model and rules for readers

The application opens the database in WAL (write-ahead log) mode with `synchronous=NORMAL`. In WAL mode readers never block the writer and the writer never blocks readers. Each CT008 instance commits one short transaction per second containing that second's readings, its status updates and heartbeats. SQLite serialises writers across processes with a file lock; each instance waits up to 5 seconds for the lock rather than failing.

A client that follows these rules will never interfere with logging:

- **Set a busy timeout** on every connection, for example `PRAGMA busy_timeout=2000;`, so a poll that coincides with a commit waits a few milliseconds instead of failing with "database is locked".
- **Keep read transactions short.** Run each query as its own autocommit statement. A read transaction left open for minutes stops the write-ahead log from being checkpointed and it grows without bound.
- **Do not write.** The client should not insert, update, delete or vacuum. In particular, do not change `journal_mode`. If a future design needs the client to delete consumed rows, use `BEGIN IMMEDIATE` transactions with a busy timeout; ask before doing so.
- **Do not create the file.** Open with a flag that fails if the file is missing (`FailIfMissing=True` in System.Data.SQLite, `mode=ro` or `mode=rw` in a URI filename). Otherwise a mistyped path silently creates an empty database.
- **Do not copy the file while it is open** by copying the .sqlite file alone: the newest data is in the -wal file. To take a snapshot use `VACUUM INTO 'copy.sqlite';` or the SQLite backup API.
- **Tolerate absence.** The file does not exist until the first CT008 instance has started. The tables are created by that instance. Retry the open once per poll interval until it succeeds.

Any SQLite library from 2015 or later handles WAL databases. The application itself uses System.Data.SQLite 1.0.119 with SQLite engine 3.46.1.

4. Schema reference

The schema version is stored in `PRAGMA user_version` and is currently **3**. See section 10 for what a version change means.

4.1 detector_status

Column	Type	Meaning
`serial`	`TEXT PRIMARY KEY`	Detector identity. The detector's serial number, or its Bluetooth MAC address without colons when no serial number is available.
`name`	`TEXT`	Name the detector advertises, for example `CT008-F`. Not unique.
`mac`	`TEXT`	Bluetooth MAC address, formatted with colons.
`firmware`	`TEXT`	Firmware version reported by the detector.
`device_type`	`INTEGER`	Instrument type number. A MicroR meter reports `14`. See section 4.3.
`session_id`	`TEXT`	Logging session the detector is currently recording into. NULL when connected but not logging, and when disconnected.
`state`	`TEXT NOT NULL`	One of `connected`, `logging`, `disconnected`. See section 6.
`response_time`	`INTEGER`	Response time in seconds as confirmed by the meter. `0` means automatic. NULL until the meter has reported it. See section 6.3.
`last_sample_ms`	`INTEGER`	Timestamp of the newest reading recorded for this detector, UTC Unix milliseconds. NULL until the first reading.
`updated_ms`	`INTEGER NOT NULL`	Heartbeat. Rewritten every second while a live CT008 instance has the detector connected, and on every state change. See section 6.2.

4.2 samples

Column	Type	Meaning
`id`	`INTEGER PRIMARY KEY AUTOINCREMENT`	Strictly increasing across all detectors and all writer instances, never reused. The client's cursor. See section 7.
`serial`	`TEXT NOT NULL`	Detector identity, matches `detector_status.serial`.
`session_id`	`TEXT NOT NULL`	Logging session this reading belongs to.
`seq`	`INTEGER NOT NULL`	Reading number within the session for this detector, starting at 1. A gap means a reading was lost between the detector and the database.
`ts_ms`	`INTEGER NOT NULL`	When the reading arrived at the PC, UTC Unix milliseconds. See section 5.3.
`count0`	`REAL`	Count rate in counts per minute, as displayed on the detector.

Column	Type	Meaning
`count0_raw`	`REAL`	Count rate in counts per minute before dead-time compensation.
`hdm`	`REAL`	High dose monitor reading in millivolts.
`hdm_raw`	`REAL`	High dose monitor reading in millivolts before zero correction.
`dose_rate`	`REAL`	Dose rate in microsieverts per hour.
`temperature`	`REAL`	Detector temperature in degrees Celsius.
`pressure`	`REAL`	Not used by the MicroR meter; always zero. Reserved for other EIC instruments that share this database format.

Indexes: `ix_samples_serial_ts` on `(serial, ts_ms)` and `ix_samples_ts` on `(ts_ms)`. Reads by `id` use the primary key.

A table named `sqlite_sequence` also exists. SQLite maintains it for the AUTOINCREMENT column; it is internal and must not be modified.

4.3 Instrument type

The MicroR meter reports `device_type` 14. The database format is shared by other EIC instruments, so a client that wants to be strict can filter on `device_type = 14` in `detector_status` and ignore any other row. In a deployment with MicroR meters only, every row will carry 14.

5. Detector identity, sessions and timestamps

5.1 Identity

Key everything by `serial`. Names are not unique: two detectors of the same model advertise the same name. A detector keeps its serial across reconnects, across CT008 restarts and across PCs.

5.2 Sessions

A session starts when an operator clicks Start Data Logging in a CT008 instance and ends at Stop or disconnect. The id is the operator's session name, reduced to letters, digits, hyphens and underscores, followed by an underscore and eight random hexadecimal characters, for example `morning-run_a3f9c21b`. Each instance and each Start click produces a new id, so several detectors logging at once on separate instances have separate sessions. Session ids do not sort chronologically; use the first reading's timestamp to order them (see section 11).

5.3 Timestamps

All timestamps are integers in milliseconds since 1970-01-01 00:00:00 UTC, taken from the PC clock at the moment the reading arrived over Bluetooth. Store and compare them as integers; convert to text only for display.

- SQL, local time: `datetime(ts_ms / 1000, 'unixepoch', 'localtime')`, or with milliseconds `strftime('%Y-%m-%d %H:%M:%f', ts_ms / 1000.0, 'unixepoch', 'localtime')`
- C#: `DateTimeOffset.FromUnixTimeMilliseconds(tsMs).ToLocalTime()`
- Python: `datetime.fromtimestamp(ts_ms / 1000, tz=timezone.utc)`

Because the client runs on the same PC, comparing `ts_ms` against the client's own clock is exact: there is no clock skew to allow for. The one-second cadence is driven by a timer in the application plus the Bluetooth round trip, so consecutive readings are roughly, not exactly, 1000 ms apart, and readings from different detectors are not aligned to each other. Section 8 explains what this means for aggregation.

6. Interpreting detector status

6.1 States

state	Meaning
`connected`	A CT008 instance is connected to the detector but logging has not been started. No rows are being written to `samples`.
`logging`	Readings are being written once per second under `session_id`.
`disconnected`	The detector disconnected, or the instance shut down cleanly. The row stays so the detector remains visible.

6.2 The heartbeat and stale rows

While an instance has a detector connected it rewrites `updated_ms` every second. If the instance crashes or is killed, its rows keep their last state but the heartbeat stops. The client must therefore treat a row whose `updated_ms` is older than a few seconds as dead, whatever its `state` says. Five seconds is a good threshold: it tolerates a busy PC without hiding a real failure for long.

A row in the `disconnected` state is not heartbeated. Its `updated_ms` records when it disconnected. Do not apply the stale rule to disconnected rows.

A detector in the `logging` state whose `last_sample_ms` stops advancing while the heartbeat continues indicates a detector that has gone quiet rather than an instance that has died.

6.3 Response time

The MicroR meter has a response time setting of 1 to 30 seconds, or 0 for automatic, which controls how long the meter averages when computing its displayed count rate. The application queries it on connect and re-reports it whenever the operator changes it, so `response_time` reflects the value the detector has actually confirmed. **For 5-second sampling the response time must be 1 second**; at any other setting each 1-second reading already contains an average over a longer period and 5-second means become smeared. The client should warn when a live detector reports anything other than 1. The CT008 application has a response time selector next to its logging controls.

7. Reading new data: the cursor pattern

Readings are appended with strictly increasing `id` values. Because SQLite allows only one writer at a time and the column is AUTOINCREMENT, an id committed later is always larger than every id committed earlier, across every writer instance, and ids are never reused after deletion. The client therefore reads new data with a single query and one remembered number:

```
SELECT id, serial, session_id, seq, ts_ms,
       count0, count0_raw, hdm, hdm_raw, dose_rate, temperature
FROM samples
WHERE id > :last_seen_id
ORDER BY id
LIMIT 5000;
```

Process the rows in order, set `last_seen_id` to the last id returned, and repeat. Polling once per second matches the writer's cadence; there is nothing to gain from polling faster because the writer commits once a second.

- **Starting point.** To show only new data from now on, seed the cursor with `SELECT COALESCE(MAX(id), 0) FROM samples;`. To resume after a client restart without missing anything, persist the cursor and continue from it. Rows older than the retention window will be gone, so a very old cursor simply skips ahead.
- **Recreated database.** If the database file is deleted and recreated, ids start again at 1 and a stored cursor would be ahead of every new row. Guard against it with one query per poll: if `SELECT MAX(id) FROM samples` is smaller than the cursor, set the cursor to that value.
- **Latest reading per detector** is a separate query, useful at startup: see section 11.
- **Batch size.** The LIMIT bounds the work per poll after a long pause. Loop until a poll returns fewer rows than the limit.
- **Late rows.** The writer batches readings and commits once a second, so a reading stamped at time T may become visible up to about a second later. Never assume that all readings with `ts_ms` below some value are already present.

> **Why not poll by timestamp?** Two readings can share a timestamp, a late commit can insert a reading with an older timestamp than one already seen, and timestamps are not unique across detectors. The id cursor has none of these problems and uses the primary key index.

8. Building 5-second samples from 1-second readings

The client's job is to turn 1-second readings into 5-second samples. Two approaches work; one does not.

8.1 Count-based: the next 5 readings (recommended for triggered samples)

When a sample is triggered, take the readings for that detector whose `ts_ms` is at or after the trigger time, in id order. Discard the first one: with a 1-second response time each reading integrates the second before its timestamp, so the first reading's period began before the trigger. Average the next 5. The sample then covers exactly the 5 seconds from the discarded reading's timestamp to the last reading's timestamp; report those two

times as the coverage. This always yields 5 readings when the meter is logging and completes about 6 seconds after the trigger. Apply a timeout, for example 12 seconds, so a meter that stops logging mid-sample produces an incomplete result rather than hanging.

8.2 Time-binned: fixed 5-second bins (for continuous aggregation)

For continuous aggregation, assign each reading to bin `ts_ms / 5000` (integer division) per detector and average per bin. This aligns bins to the wall clock, which is convenient for correlating detectors. Because of the timing jitter described in section 5.3, a bin holds 4, 5 or 6 readings. Store the count per bin alongside the mean and do not assume 5. Close a bin only when a reading from a later bin has arrived, or when the bin is more than about 2 seconds old, to allow for the writer's one-second commit delay.

8.3 What not to do

Do not open a 5.000-second stopwatch at the trigger and average whatever falls inside it. The same jitter means such a window regularly captures 4 readings, and occasionally 6, and the result is not comparable from one sample to the next. This was verified during development and is why the count-based method above is recommended.

8.4 Averaging

All measured columns are rates or levels, so a plain arithmetic mean is correct for each. Use `count0` for the displayed count rate and `dose_rate` for dose rate. If the client needs counts rather than a rate, `count0 / 60` is the number of counts in that one-second reading when the response time is 1 second. Check `seq` for gaps if a sample must be rejected when a reading was lost.

9. Data retention

Each running CT008 instance deletes readings older than `SqliteRetentionHours` (default 24) once a minute. Rows in `detector_status` are never deleted. The purge only runs while an instance is running, and the file does not shrink on disk after deletion; SQLite reuses the freed space, so the file settles at roughly one retention window of data.

Consequences for the client:

- The database is a live feed with a short memory, not an archive. Anything the client needs long-term must be copied out within the retention window. In steady operation the client processes each reading within a second of its arrival, so this only matters after an outage.
- If the client can be offline for longer than the retention window, either raise `SqliteRetentionHours` in the CT008 configuration or accept the gap. Do not have the client delete rows itself unless that is agreed, so that exactly one party owns deletion.

10. Schema versioning and forward compatibility

`PRAGMA user_version` identifies the schema. The application upgrades an existing file in place when it starts with a newer schema, inside the same locked transaction that creates the tables, so the client never sees a half-

upgraded database.

Version	Change
1	Initial schema.
2	Removed the `pid` column from `detector_status`.
3	Added `response_time` to `detector_status`.

To stay compatible with future versions: check that `user_version` is at least 3 and refuse to run against an older file; read columns by name rather than by position; ignore columns you do not know. Existing columns will not change meaning or type. Removals or renames will be announced in a revision of this guide with a version bump.

11. Useful queries

Detectors and their status with readable times and a staleness flag:

```
SELECT serial, name, state, session_id, response_time,
       datetime(updated_ms / 1000, 'unixepoch', 'localtime')      AS heartbeat,
       datetime(last_sample_ms / 1000, 'unixepoch', 'localtime')  AS last_reading,
       CASE WHEN state <> 'disconnected'
             AND (strftime('%s','now') * 1000 - updated_ms) > 5000
            THEN 1 ELSE 0 END                                       AS stale
FROM detector_status
ORDER BY serial;
```

Newest reading per detector, for populating a display at startup:

```
SELECT s.*
FROM samples s
WHERE s.id IN (SELECT MAX(id) FROM samples GROUP BY serial);
```

Sessions with their time range and reading count:

```
SELECT session_id, serial,
       datetime(MIN(ts_ms) / 1000, 'unixepoch', 'localtime') AS started,
       datetime(MAX(ts_ms) / 1000, 'unixepoch', 'localtime') AS last_reading,
       COUNT(*)                                              AS readings,
       MAX(seq) - COUNT(*)                                   AS missing_readings
FROM samples
GROUP BY session_id, serial
ORDER BY MIN(ts_ms);
```

Fixed 5-second bins for one detector over the last ten minutes:

```
SELECT (ts_ms / 5000) * 5000                          AS bin_start_ms,
       COUNT(*)                                        AS readings,
       AVG(count0)                                     AS mean_cpm,
       AVG(dose_rate)                                  AS mean_dose_rate,
       AVG(temperature)                                AS mean_temperature
FROM samples
WHERE serial = :serial
  AND ts_ms >= strftime('%s','now') * 1000 - 600000
GROUP BY ts_ms / 5000
ORDER BY bin_start_ms;
```

12. Code examples

12.1 C# with System.Data.SQLite

```
using System.Data.SQLite;

var builder = new SQLiteConnectionStringBuilder
{
    DataSource = @"C:\Program Files\EIC\CT008\Database\ct008-log.sqlite",
    Version = 3,
    Pooling = false,
    FailIfMissing = true      // never create an empty database at a wrong path
};

using (var conn = new SQLiteConnection(builder.ToString()))
{
    conn.Open();
    using (var cmd = new SQLiteCommand("PRAGMA busy_timeout=2000;", conn))
        cmd.ExecuteNonQuery();

    long lastSeen;
    using (var cmd = new SQLiteCommand("SELECT COALESCE(MAX(id), 0) FROM samples;", conn))
        lastSeen = Convert.ToInt64(cmd.ExecuteScalar());

    while (running)
    {
        using (var cmd = new SQLiteCommand(
            "SELECT id, serial, session_id, seq, ts_ms, count0, count0_raw, hdm, hdm_raw, " +
            "dose_rate, temperature FROM samples WHERE id > @id ORDER BY id LIMIT 5000;", conn))
        {
            cmd.Parameters.AddWithValue("@id", lastSeen);
            using (SQLiteDataReader r = cmd.ExecuteReader())
            {
                while (r.Read())
                {
                    lastSeen = r.GetInt64(r.GetOrdinal("id"));
                    string serial = r.GetString(r.GetOrdinal("serial"));
                    long tsMs = r.GetInt64(r.GetOrdinal("ts_ms"));
                    double cpm = r.GetDouble(r.GetOrdinal("count0"));
                    HandleReading(serial, tsMs, cpm /* ... */);
                }
            }
        }
```

```
        Thread.Sleep(1000);
    }
}
```

Keep one connection open for the life of the poller rather than opening one per poll. Wrap the poll body in a try/catch: on any SQLiteException, log it, close and reopen the connection on the next iteration, and keep the cursor.

12.2 Python with the standard library

```
import sqlite3, time
from datetime import datetime, timezone

path = r"C:\Program Files\EIC\CT008\Database\ct008-log.sqlite"
conn = sqlite3.connect(f"file:{path}?mode=ro", uri=True, timeout=2.0)
conn.row_factory = sqlite3.Row

last_seen = conn.execute("SELECT COALESCE(MAX(id), 0) FROM samples").fetchone()[0]

while True:
    rows = conn.execute(
        "SELECT id, serial, session_id, seq, ts_ms, count0, count0_raw, hdm, hdm_raw, "
        "dose_rate, temperature FROM samples WHERE id > ? ORDER BY id LIMIT 5000",
        (last_seen,)).fetchall()
    for r in rows:
        last_seen = r["id"]
        when = datetime.fromtimestamp(r["ts_ms"] / 1000, tz=timezone.utc)
        handle_reading(r["serial"], when, r["count0"])
    time.sleep(1)
```

Opening read-only (`mode=ro`) is safe here because a CT008 instance is normally running and has already created the -wal and -shm files. If the client may start before any instance, use `mode=rw` instead and simply never write.

13. Troubleshooting

Symptom	Cause and remedy
"database is locked"	No busy timeout set, or a long-running read transaction elsewhere. Set `busy_timeout`; keep transactions short.
"no such table: samples"	The file exists but no CT008 instance has created the schema yet, or the client created an empty file by opening a wrong path. Check the path and open with fail-if-missing.
"unable to open database file"	Path wrong, the client lacks permission to the folder, or the file is on a network share.
The -wal file keeps growing	A reader is holding a transaction open, which prevents checkpoints. Find the long-lived transaction; DB Browser for SQLite with a pending edit is a common culprit during testing.
A detector shows `logging` but no new readings arrive	Compare `updated_ms` with the clock. If it is stale, the CT008 instance has died; if it is fresh, the detector itself has stopped responding.

Symptom	Cause and remedy
5-second means look smoothed or lag behind	Check `response_time`; it must be 1 for 5-second samples.
Readings appear a second late	Expected: the writer commits once per second. Design around `id`, not timestamps.
Old readings disappeared	Retention purge, default 24 hours. See section 9.

14. Reference implementation

EIC's CT008 Monitor is a small Windows application that implements everything in this guide: opening and re-opening the database, the id cursor, detector rows with staleness detection, the response time warning and triggered 5-reading samples. Its `DatabaseReader.cs` is a self-contained read-only access class in C# and can be reused directly. It targets .NET Framework 4.8 and the same SQLite library as the CT008 application. Contact EIC for the source.

For ad hoc inspection during development, DB Browser for SQLite (free, sqlitebrowser.org) opens the live database. Use File, Open Database Read Only so that it never holds a write lock, and apply a custom display format of `datetime("ts_ms" / 1000, 'unixepoch', 'localtime')` to timestamp columns to read them as local time.